



Principal Component Analysis (PCA) on Wine Dataset

Practical Exercise

Objective

In this practical, we will learn how to use **Principal Component Analysis (PCA)** for dimensionality reduction using the Wine dataset.

We will understand:

- Why dimensionality reduction is needed
- Why feature scaling is important before PCA
- What Principal Components are
- Explained variance and cumulative explained variance
- How to select the number of components
- How to visualize high-dimensional data in 2D
- How to interpret PCA loadings

Important: PCA does not use the target variable to create Principal Components. The target is used only later for visualization.

1. Data Collection and Data Preprocessing

Before applying PCA, we first understand and prepare the dataset.

The main steps are:

1. Import the libraries and dataset
2. Understand the dataset
3. Check for null values
4. Check for duplicate values
5. Visualize the data
6. Detect outliers
7. Select features for PCA
8. Perform feature scaling

1.1 Importing the Libraries and Dataset

We will use the **Wine dataset** available in Scikit-learn.

The dataset contains **178 observations, 13 numerical features, and 3 wine classes**.

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import load_wine

wine = load_wine(as_frame=True)
df = wine.frame

df.head()
```

```
Out[1]:
```

	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium	total_phenols	flavanoids	nonflava
0	14.23	1.71	2.43	15.6	127.0	2.80	3.06	
1	13.20	1.78	2.14	11.2	100.0	2.65	2.76	
2	13.16	2.36	2.67	18.6	101.0	2.80	3.24	
3	14.37	1.95	2.50	16.8	113.0	3.85	3.49	
4	13.24	2.59	2.87	21.0	118.0	2.80	2.69	

1.2 Understanding the Dataset

Let us examine the shape, column names, data types, and statistical summary.

This helps us understand the structure of the dataset before applying PCA.

```
In [2]: print("Shape of dataset:", df.shape)

print("\nColumn Names:")
print(df.columns)

print("\nData Types:")
print(df.dtypes)

print("\nStatistical Summary:")
df.describe()
```

Shape of dataset: (178, 14)

Column Names:

```
Index(['alcohol', 'malic_acid', 'ash', 'alcalinity_of_ash', 'magnesium',  
      'total_phenols', 'flavanoids', 'nonflavanoid_phenols',  
      'proanthocyanins', 'color_intensity', 'hue',  
      'od280/od315_of_diluted_wines', 'proline', 'target'],  
      dtype='str')
```

Data Types:

```
alcohol          float64  
malic_acid       float64  
ash              float64  
alcalinity_of_ash float64  
magnesium        float64  
total_phenols    float64  
flavanoids       float64  
nonflavanoid_phenols float64  
proanthocyanins  float64  
color_intensity  float64  
hue              float64  
od280/od315_of_diluted_wines float64  
proline          float64  
target           int64  
dtype: object
```

Statistical Summary:

Out[2]:

	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium	total_phenols	flava
count	178.000000	178.000000	178.000000	178.000000	178.000000	178.000000	178.0
mean	13.000618	2.336348	2.366517	19.494944	99.741573	2.295112	2.0
std	0.811827	1.117146	0.274344	3.339564	14.282484	0.625851	0.9
min	11.030000	0.740000	1.360000	10.600000	70.000000	0.980000	0.3
25%	12.362500	1.602500	2.210000	17.200000	88.000000	1.742500	1.2
50%	13.050000	1.865000	2.360000	19.500000	98.000000	2.355000	2.1
75%	13.677500	3.082500	2.557500	21.500000	107.000000	2.800000	2.8
max	14.830000	5.800000	3.230000	30.000000	162.000000	3.880000	5.0

Observation

The Wine dataset contains **13 input features** and **1 target column**.

The input features describe chemical properties of wine. The target identifies the wine class.

For PCA, we will use only the 13 input features.

1.3 Checking for Null Values

Missing values can affect mathematical calculations, so we check for null values before applying PCA.

```
In [3]: df.isnull().sum()
```

```
Out[3]: alcohol                0
malic_acid                    0
ash                           0
alcalinity_of_ash             0
magnesium                     0
total_phenols                 0
flavanoids                    0
nonflavanoid_phenols          0
proanthocyanins               0
color_intensity               0
hue                           0
od280/od315_of_diluted_wines 0
proline                       0
target                        0
dtype: int64
```

Observation

If all values are 0, the dataset contains no missing values and no missing-value treatment is required.

1.4 Checking for Duplicate Values

Duplicate records may affect our analysis. Therefore, we check for duplicate rows.

```
In [4]: print("Number of duplicate rows:", df.duplicated().sum())
```

```
Number of duplicate rows: 0
```

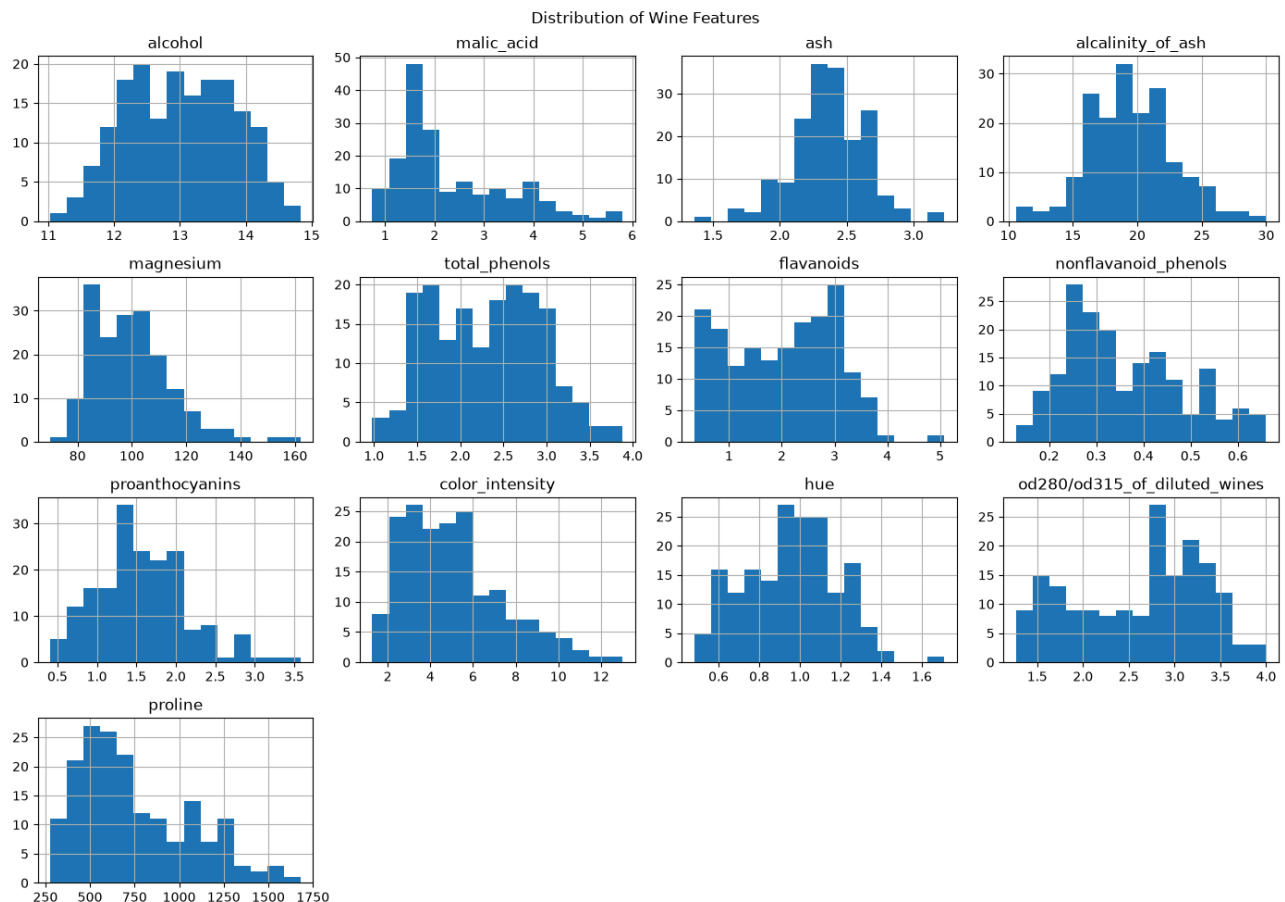
Observation

If the result is 0, there are no duplicate records in the dataset.

1.5 Data Visualization

The Wine dataset contains many numerical features. We can use histograms to understand their distributions and ranges.

```
In [5]: df[wine.feature_names].hist(figsize=(14, 10), bins=15)
plt.suptitle("Distribution of Wine Features")
plt.tight_layout()
plt.show()
```



Observation

The features have very different numerical ranges.

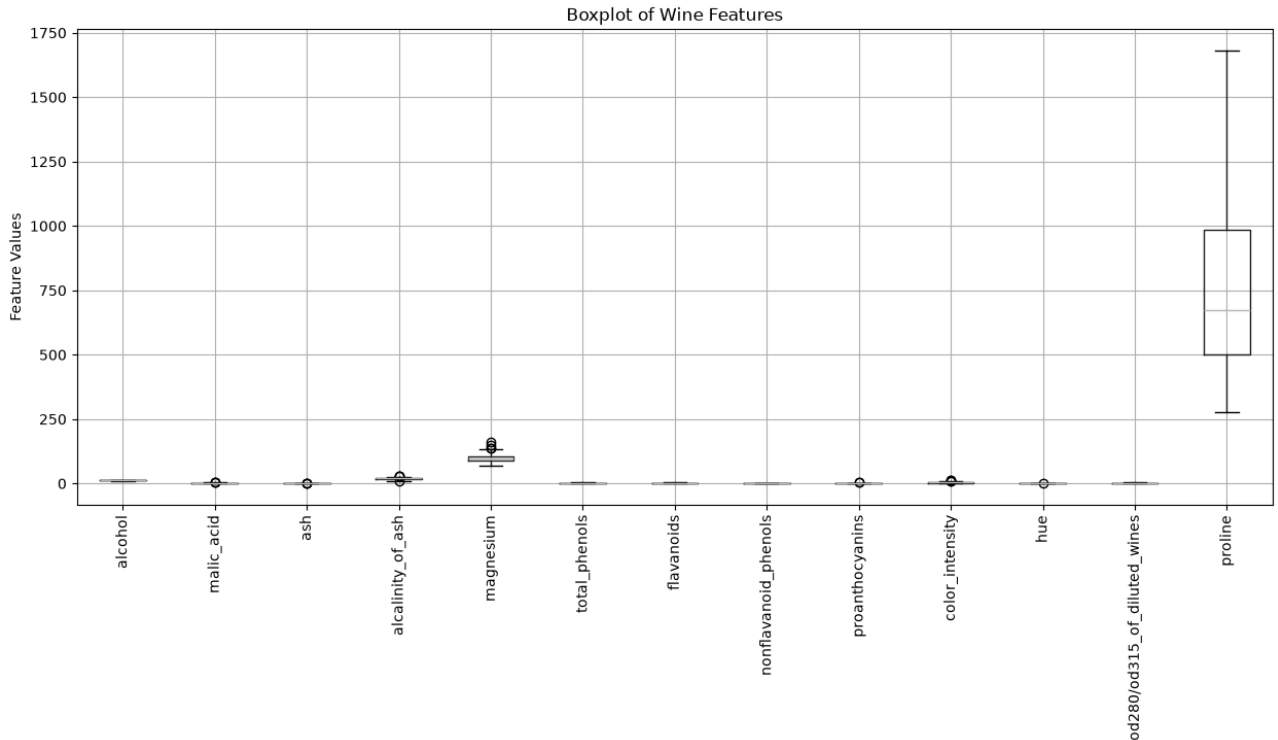
Because PCA is based on variance, features with larger scales can dominate the analysis. Therefore, **feature scaling is important before PCA**.

1.6 Outlier Detection

A boxplot helps us identify observations that appear unusually far from the main distribution.

An unusual observation is not automatically an error, so we should not remove it without a proper reason.

```
In [6]: plt.figure(figsize=(15, 6))
df[wine.feature_names].boxplot()
plt.xticks(rotation=90)
plt.title("Boxplot of Wine Features")
plt.ylabel("Feature Values")
plt.show()
```



1.7 Selecting Features for PCA

Now we separate the input features from the target.

The **13 numerical features** will be used for PCA.

The target is kept separately only so that we can later color our visualization according to wine class.

Important: The target is NOT used to calculate the Principal Components.

```
In [7]: X = df[wine.feature_names]
y = df["target"]

print("Input feature shape:", X.shape)
print("Target shape:", y.shape)
```

Input feature shape: (178, 13)
Target shape: (178,)

1.8 Feature Scaling

PCA is variance-based. If one feature has a much larger numerical range than another, it can have an unnecessarily large influence on the Principal Components.

Therefore, we standardize the features using `StandardScaler`.

After scaling:

- Mean ≈ 0
- Standard deviation ≈ 1

```
In [8]: from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_scaled = pd.DataFrame(X_scaled, columns=wine.feature_names)

X_scaled.head()
```

```
Out[8]:
```

	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium	total_phenols	flavanoids	no
0	1.518613	-0.562250	0.232053	-1.169593	1.913905	0.808997	1.034819	
1	0.246290	-0.499413	-0.827996	-2.490847	0.018145	0.568648	0.733629	
2	0.196879	0.021231	1.109334	-0.268738	0.088358	0.808997	1.215533	
3	1.691550	-0.346811	0.487926	-0.809251	0.930918	2.491446	1.466525	
4	0.295700	0.227694	1.840403	0.451946	1.281985	0.808997	0.663351	

Observation

The features are now on a comparable scale.

We are ready to apply PCA.

2. Principal Component Analysis (PCA)

2.1 What is PCA?

Principal Component Analysis (PCA) is an unsupervised dimensionality reduction technique.

It transforms the original features into new variables called **Principal Components**.

Simple idea

If a dataset contains 13 features, PCA may allow us to represent most of the important variation using fewer dimensions.

Original Features → PCA → Principal Components

2.2 Why Do We Need PCA?

High-dimensional datasets can create several challenges:

- Visualization becomes difficult
- Computational cost can increase
- Some features may contain redundant information
- Relationships among many features become difficult to understand

PCA helps us create a smaller representation while retaining as much important variance as possible.

Classroom Analogy

Imagine a 100-page book. Instead of reading every page, we create a short summary containing the most important information.

PCA performs a similar idea for numerical data: it creates a lower-dimensional representation that preserves important variation.

2.3 What is a Principal Component?

A **Principal Component** is a new variable created as a combination of the original features.

The components are ordered according to the amount of variance they capture.

- **PC1** captures the maximum possible variance.
- **PC2** captures the next highest variance while being orthogonal to PC1.
- **PC3** captures the next highest remaining variance.
- The process continues for the remaining components.

2.4 How Does PCA Work?

The basic process is:

1. Standardize the features.
2. Find directions in which the data varies the most.
3. Select the direction with maximum variance as PC1.
4. Find the next important direction as PC2.
5. Continue for additional components.
6. Transform the original data into the component space.
7. Keep the required number of components.

High-dimensional data → PCA → Low-dimensional data

2.5 Understanding Variance in PCA

PCA tries to find directions that capture high variance.

The **explained variance** tells us how much variation is captured by a component.

The **explained variance ratio** tells us the proportion of total variance captured by each component.

2.6 Applying PCA with All Components

First, we apply PCA without reducing the number of components.

Since there are 13 original features, PCA can produce up to 13 Principal Components.

This allows us to examine how variance is distributed among the components.

```
In [9]: from sklearn.decomposition import PCA

pca = PCA()
X_pca = pca.fit_transform(X_scaled)

print("Original shape:", X_scaled.shape)
print("PCA shape:", X_pca.shape)
```

```
Original shape: (178, 13)
PCA shape: (178, 13)
```

Observation

The shape remains the same because all 13 components were retained.

Our next step is to determine how much variance each component captures.

2.7 Explained Variance Ratio

The explained variance ratio tells us how much of the total variance is represented by each Principal Component.

```
In [10... explained_variance = pca.explained_variance_ratio_

explained_variance_df = pd.DataFrame({
    "Principal Component": range(1, len(explained_variance) + 1),
    "Explained Variance Ratio": explained_variance,
    "Explained Variance (%)": explained_variance * 100
})

explained_variance_df
```

Out[10]:

	Principal Component	Explained Variance Ratio	Explained Variance (%)
0	1	0.361988	36.198848
1	2	0.192075	19.207490
2	3	0.111236	11.123631
3	4	0.070690	7.069030
4	5	0.065633	6.563294
5	6	0.049358	4.935823
6	7	0.042387	4.238679
7	8	0.026807	2.680749
8	9	0.022222	2.222153
9	10	0.019300	1.930019
10	11	0.017368	1.736836
11	12	0.012982	1.298233
12	13	0.007952	0.795215

2.8 Cumulative Explained Variance

Cumulative explained variance tells us how much total variance is retained when the first several components are combined.

For example:

- PC1 → variance captured by PC1
- PC1 + PC2 → total variance captured by the first two components
- PC1 + PC2 + PC3 → total variance captured by the first three components

This helps us decide how many components should be retained.

```
In [11... cumulative_variance = np.cumsum(pca.explained_variance_ratio_)

cumulative_variance_df = pd.DataFrame({
    "Number of Components": range(1, len(cumulative_variance) + 1),
    "Cumulative Explained Variance": cumulative_variance,
    "Cumulative Explained Variance (%)": cumulative_variance * 100
})

cumulative_variance_df
```

Out[11]:

	Number of Components	Cumulative Explained Variance	Cumulative Explained Variance (%)
0	1	0.361988	36.198848
1	2	0.554063	55.406338
2	3	0.665300	66.529969
3	4	0.735990	73.598999
4	5	0.801623	80.162293
5	6	0.850981	85.098116
6	7	0.893368	89.336795
7	8	0.920175	92.017544
8	9	0.942397	94.239698
9	10	0.961697	96.169717
10	11	0.979066	97.906553
11	12	0.992048	99.204785
12	13	1.000000	100.000000

3. Choosing the Number of Principal Components

3.1 Visualizing Cumulative Explained Variance

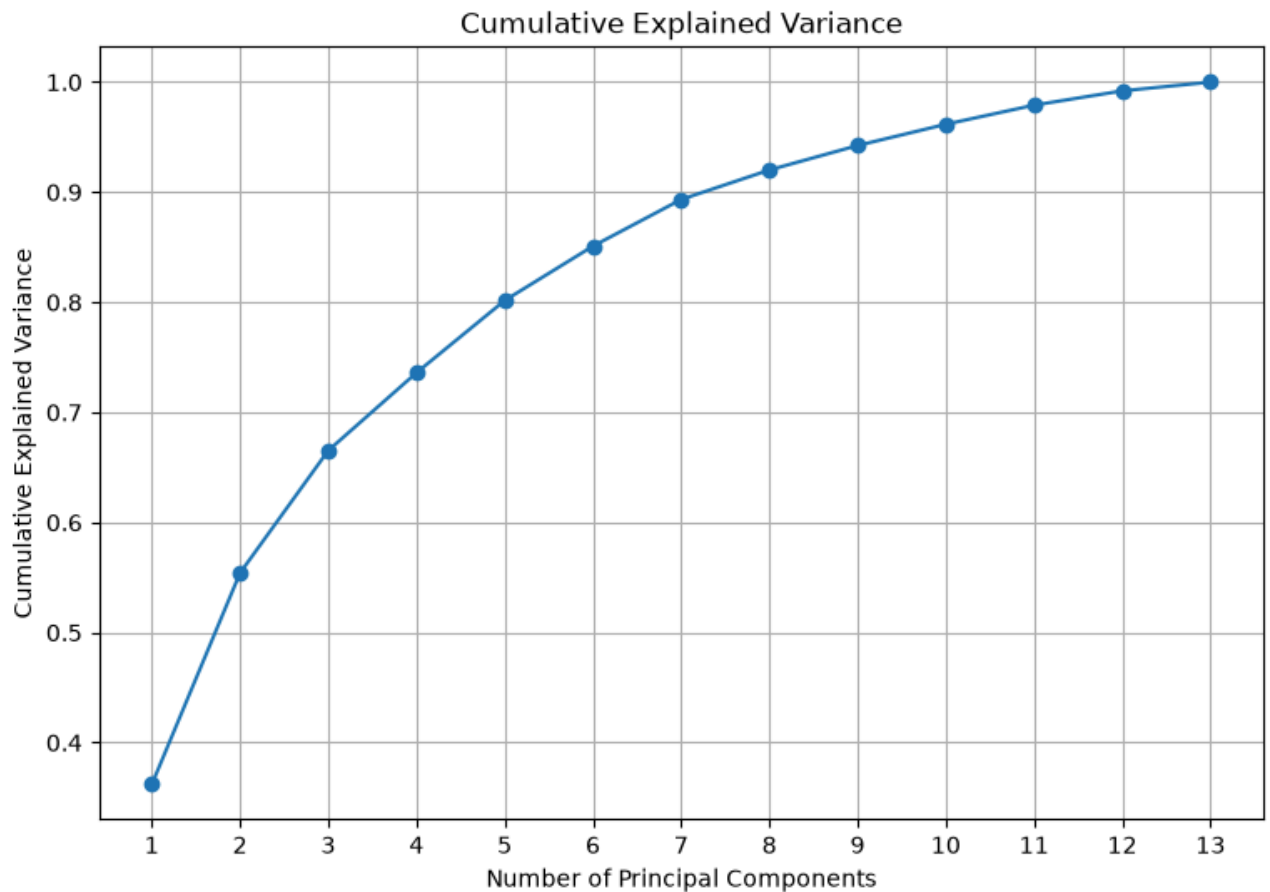
A cumulative variance plot helps us determine how many components are sufficient.

We look for a point where additional components contribute relatively little additional variance.

```
In [12... plt.figure(figsize=(9, 6))

plt.plot(
    range(1, len(cumulative_variance) + 1),
    cumulative_variance,
    marker="o"
)

plt.xlabel("Number of Principal Components")
plt.ylabel("Cumulative Explained Variance")
plt.title("Cumulative Explained Variance")
plt.xticks(range(1, len(cumulative_variance) + 1))
plt.grid(True)
plt.show()
```



Question

If the first few components capture most of the variance, do we really need all 13 original features?

This is the main motivation behind dimensionality reduction.

3.2 Selecting Components Based on Explained Variance

A common approach is to retain a desired percentage of variance, such as:

- 80%
- 90%
- 95%

Common Choices

Variance Retained	When It May Be Useful
80%	Strong dimensionality reduction when some information loss is acceptable
90%	Good balance between dimensionality reduction and information retention
95%	Common choice when we want to preserve most of the important information
99%	Useful when information loss should be very small

Let us find the number of components required to retain approximately **95% of the variance**.

```
In [13... n_components_95 = np.argmax(cumulative_variance >= 0.95) + 1
print("Number of components required for 95% variance:",
      n_components_95)
print("Variance retained:",
      cumulative_variance[n_components_95 - 1] * 100, "%")
```

```
Number of components required for 95% variance: 10
Variance retained: 96.16971684450644 %
```

Observation

The required number of components depends on the dataset.

If only a few components retain most of the variance, PCA has successfully reduced dimensionality while preserving most of the variation.

4. Applying PCA with Selected Components

4.1 Creating the Reduced Dataset

Now we apply PCA using the number of components selected from the explained variance analysis.

This creates a lower-dimensional representation of the Wine dataset.

```
In [14... pca_reduced = PCA(n_components=n_components_95)
X_reduced = pca_reduced.fit_transform(X_scaled)
```

```
print("Original data shape:", X_scaled.shape)
print("Reduced data shape:", X_reduced.shape)
```

Original data shape: (178, 13)
Reduced data shape: (178, 10)

Observation

The original dataset contains 13 features.

The reduced dataset contains fewer Principal Components while retaining approximately 95% of the original variance.

This is **dimensionality reduction**.

4.2 PCA for 2D Visualization

We can also reduce the data to only two components.

This allows us to visualize the high-dimensional dataset on a 2D graph.

- PC1 → X-axis
- PC2 → Y-axis

The target class is used only for coloring the points.

Important: The target is not used to fit PCA.

```
In [15... pca_2d = PCA(n_components=2)
X_pca_2d = pca_2d.fit_transform(X_scaled)

pca_2d_df = pd.DataFrame(
    X_pca_2d,
    columns=["PC1", "PC2"]
)

pca_2d_df["Target"] = y.values

pca_2d_df.head()
```

```
Out[15]:
```

	PC1	PC2	Target
0	3.316751	1.443463	0
1	2.209465	-0.333393	0
2	2.516740	1.031151	0
3	3.757066	2.756372	0
4	1.008908	0.869831	0

5. Visualizing the PCA Results

5.1 Plotting PC1 vs PC2

Let us visualize the Wine samples using the first two Principal Components.

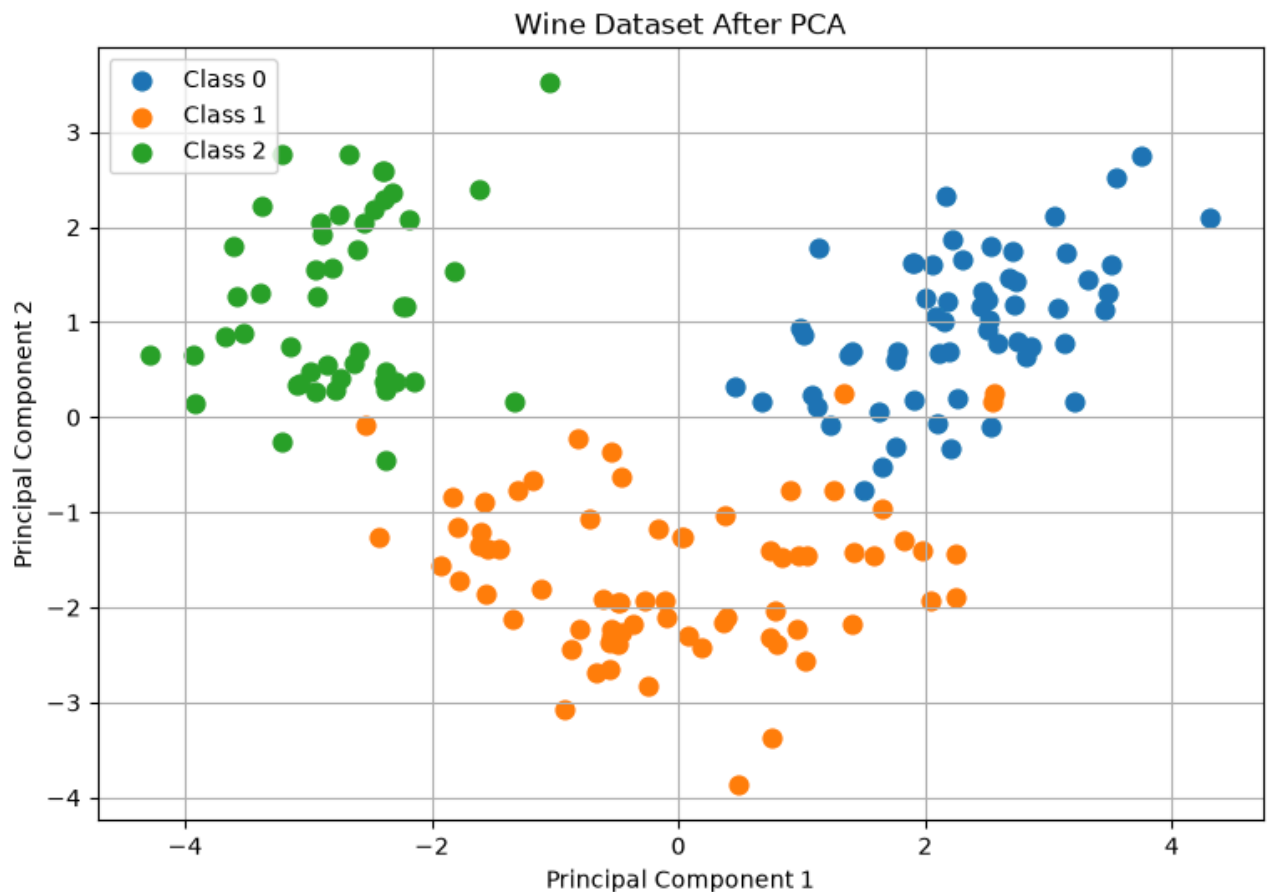
Different colors represent the different wine classes.

```
In [16... plt.figure(figsize=(9, 6))

for target_class in sorted(pca_2d_df["Target"].unique()):
    class_data = pca_2d_df[pca_2d_df["Target"] == target_class]

    plt.scatter(
        class_data["PC1"],
        class_data["PC2"],
        label=f"Class {target_class}",
        s=60
    )

plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("Wine Dataset After PCA")
plt.legend()
plt.grid(True)
plt.show()
```

Observation

The PCA plot shows whether the wine classes appear visually separated in the new two-dimensional space.

Although the original dataset has 13 features, PCA allows us to visualize important variation using only two dimensions.

6. Understanding PCA Components

6.1 What are PCA Loadings?

PCA components are combinations of the original features.

Loadings show how strongly each original feature contributes to a Principal Component.

A large positive or negative loading indicates a stronger contribution to that component.

```
In [17]: loadings = pd.DataFrame(
    pca_2d.components_.T,
    index=wine.feature_names,
    columns=["PC1", "PC2"]
)

loadings
```

```
Out[17]:
```

	PC1	PC2
alcohol	0.144329	0.483652
malic_acid	-0.245188	0.224931
ash	-0.002051	0.316069
alcalinity_of_ash	-0.239320	-0.010591
magnesium	0.141992	0.299634
total_phenols	0.394661	0.065040
flavanoids	0.422934	-0.003360
nonflavanoid_phenols	-0.298533	0.028779
proanthocyanins	0.313429	0.039302
color_intensity	-0.088617	0.529996
hue	0.296715	-0.279235
od280/od315_of_diluted_wines	0.376167	-0.164496
proline	0.286752	0.364903

6.2 Interpreting the Loadings

When interpreting loadings:

- Large positive value → strong positive contribution
- Large negative value → strong negative contribution
- Value close to zero → smaller contribution

1. Which feature has the largest absolute loading for PC1?
2. Which feature has the largest absolute loading for PC2?
3. Are the important features the same for both components?
4. What might each Principal Component represent?

7. PCA and Dimensionality Reduction

7.1 Original Data vs Reduced Data

Before PCA:

13 original features

After PCA:

Fewer Principal Components

The Principal Components are combinations of the original variables.

Therefore, PCA changes the representation of the data rather than simply selecting a few original columns.

7.2 Is Information Lost During PCA?

Usually, some information is lost when fewer components are retained.

However, the objective is to remove relatively less important variation while preserving most of the important variance.

For example, retaining 95% explained variance means the selected components represent approximately 95% of the dataset's total variance.

So PCA involves a trade-off:

Fewer dimensions ↔ Some information loss

8. PCA vs Feature Selection

Feature Selection	PCA
Selects original features	Creates new features
Original feature meaning is retained	Components may be harder to interpret
Some columns are removed	Information is combined into components
Example: select 5 out of 13 features	Example: create PC1, PC2, PC3...

Simple Memory Trick

Feature Selection = Choose existing features

PCA = Create new features from existing features

PCA should not automatically be applied to every dataset. First understand whether dimensionality reduction is useful for the problem.

10. Student Exercises

10.1 Experiment with Different Numbers of Components

Apply PCA with:

- `n_components=2`
- `n_components=3`
- `n_components=5`

Compare the resulting data shapes.

Question: What happens when the number of components is increased?

10.2 Compare Different Variance Thresholds

Find the number of components required to retain:

- 80% variance
- 90% variance
- 95% variance

Question: How does the required number of components change as the required variance increases?

10.3 Visualize Different PCA Combinations

Create scatter plots for:

- PC1 vs PC2

- PC1 vs PC3
- PC2 vs PC3

Question: Which pair provides the clearest visual separation between the wine classes?

10.4 Analyze PCA Loadings

Examine the loading values for PC1 and PC2.

Questions:

1. Which original feature contributes most strongly to PC1?
2. Which original feature contributes most strongly to PC2?
3. Are the important features the same for both components?
4. What might each Principal Component represent?

11. Final Takeaways

What did we learn?

- **PCA** is an unsupervised dimensionality reduction technique.
- PCA transforms original features into **Principal Components**.
- **PC1 captures maximum variance**.
- Each subsequent component captures additional variance while being orthogonal to previous components.
- **Feature scaling is important before PCA**.
- **Explained variance ratio** tells us how much variance each component captures.
- **Cumulative explained variance** helps us select the number of components.
- PCA can reduce many dimensions into a smaller number of components.
- PCA can make high-dimensional data easier to visualize.
- **PCA does not use the target variable** to create components.
- PCA creates new features, whereas feature selection chooses existing features.

In []: